The Critical-Projects List Contains Kubernetes and Misses zlib

Dependency concentration as a criticality signal

Author. Chris Brink (independent) **Version.** Draft v0.5, 2026-09-02 (v0.2: retitled; artifacts section. v0.3: package-versus-repository distinction made explicit; recommendation section ends on the artifact; CLI gained direct Cargo.lock support. v0.4: tool and rankings split to their own repo, github.com/thefalsework/conemass, with commit history intact; the tool is named conemass — the metric keeps the name ORACLE in this text. v0.5: registered RustSec pooled-retrodiction result added to limitations — ORACLE lost the pooled cell to dependent count, reported in full). All computations cited here are committed with their code and raw output in `oracle-scanner/` at github.com/thefalsework/papers; each script states its expectations in a header written before the run and its results in a dated postscript.

Summary

The OpenSSF criticality-score top-1000 — the list consumed by the Securing Critical Projects working group — contains Kubernetes and misses zlib. It also misses serde, syn, proc-macro2, libexpat, and libxml2, and its collection pipeline, which enumerates GitHub-hosted source repositories, could not have ranked the xz project at any position in 2022, because xz was hosted elsewhere. One distinction runs through this piece and is best fixed now: the metric we describe ranks *shipped packages*, the objects dependency graphs contain — the Debian binary package liblzma5, the crate unicode-ident — while the incumbent ranks *upstream source repositories*. Our comparison maps packages to their upstream repositories by hand, and the two xz facts below are facts about two different objects: the package shipping the backdoored code ranked eighth in its archive by concentration, and the project's repository sat entirely outside the incumbent pipeline's coverage. We describe a complementary metric, computable from a dependency graph alone, whose top ranks are precisely the packages the incumbent misses: on the last Debian release before the xz backdoor, it ranks liblzma5 eighth in the archive, against #173 by dependent count; on crates.io it ranks unicode-ident — six direct dependents, one maintainer, present in nearly every Rust build — second in the registry, against #3,582 by dependent count. The metric is not a replacement for criticality scoring. It measures a different quantity — load rather than fame — and the two disagree exactly where prioritization mistakes are most expensive.

The metric

For a node x in a dependency graph, define

ORACLE(x) = Σ 1/|cone(u)|,

summed over all packages u whose truncated transitive-dependency set ("cone," breadth-first, capped at 200 nodes) contains x. In words: count every toolchain that x is part of, weighting each by the reciprocal of its size. A package that constitutes half of ten small toolchains scores higher than a

package that is a thousandth of ten thousand large ones, at identical dependent counts. The quantity measured is *concentration of reach*.

Two properties matter for this audience. First, the metric requires only the dependency graph — no stars, contributors, commit activity, funding data, or hosting-platform API access — so a registry operator can compute it for an entire ecosystem in minutes (our runs: 63,436 Debian packages or 84,439 crates, single-threaded, under fifteen seconds each). Second, it is structurally different from every input the incumbent uses: dependent counts, PageRank-style measures, and the criticality score's signals all reward volume or visibility, and rank-correlate accordingly.

The functional was originally derived as the closed-form expected-gain law of a synthetic graph-growth model, where it provably and completely accounts for growth differences that degree, age, PageRank, k-core, and exact transitive-dependent counts cannot express. That derivation is documented elsewhere in the repository; nothing in this piece depends on it. Here the metric is offered as a ranking, and a ranking is judged by what it flags.

Data and vintage

Three datasets, and the dating is the methodological core of the piece:

- **Debian**, main/binary-amd64 dependency graphs for the ten stable releases 2007–2025, extracted from archived Packages files with parsing choices fixed in the extraction script. The retrodiction below uses **bookworm (2023) — the last stable release before CVE-2024-3094 was disclosed in March 2024.**
- **crates.io**, the 2022 dependency snapshot from the same repository's earlier growth studies.
- **The incumbent list**: the "1000 critical projects" CSV produced by the OpenSSF criticality-score pipeline (Pike scoring), **June 2022 vintage**, retrieved from a Scorecard maintainer's archival repository.[1]

The June 2022 list and the 2022 crates snapshot are contemporaneous, and both predate the public disclosure of the xz backdoor by roughly 21 months. No ranking reported here can be contaminated by post-incident attention: the stars, mentions, and activity that xz, liblzma, and unicode-ident accumulated after March 2024 do not exist in any of this data. We did not select an old list to disadvantage the incumbent; we selected the list that removes hindsight from both sides of the comparison.

Result 1: the xz retrodiction

On Debian bookworm, one release before disclosure:

package	ORACLE rank	dependent-count rank	PageRank rank
liblzma5	**8** / 63,436	173	36
libgcrypt20	49	150	151
libexpat1	38	102	56
zlib1g	4	7	8
libssl3	18	19	35

The pattern, not the single number, is the result. Packages that are famous as well as load-bearing (zlib, OpenSSL) rank high on every metric; the metrics agree where fame is deserved. They diverge on the quiet rows: liblzma, libgcrypt, and libexpat — each a small, low-visibility library with a history of under-resourcing — move up by one to two orders of magnitude under concentration weighting. The liblzma ranking is robust to the one free parameter: at cone caps of 50, 100, 200, 400, and 800 its rank is #8 in every case, and the archive's ORACLE top-100 overlaps 92–99% between adjacent caps.

Transitive-dependent *counts*, for comparison, are unusable at the head of the distribution: hundreds of major libraries saturate the cap and tie. Concentration weighting is what separates them.

Result 2: crates.io, and a specific threat class

The same computation on the 2022 crates.io graph:

crate	ORACLE rank	dependent-count rank	direct dependents
libc	1 / 84,439	14	4,330
unicode-ident	**2**	3,582	6
proc-macro2	3	17	3,392
quote	4	16	4,135
syn	5	15	4,202
cfg-if	6	52	835
serde	7	1	16,012

unicode-ident is the illustrative row: a single-maintainer crate with six direct dependents that reaches nearly every Rust build through the proc-macro2/syn chain. Dependent-count scoring places it below three and a half thousand other crates. Concentration places it second in the registry. This is the same profile as liblzma — minimal direct visibility, near-total indirect presence — identified by the same computation in an unrelated ecosystem.

The extreme divergers form a coherent class rather than noise. Sorting the ORACLE top-1000 by how much worse their dependent-count rank is yields, almost without exception, degree-one procedural-macro companion crates: openssl-macros, wasm-bindgen-macro, pin-project-internal, darling_macro, and so on. Each has exactly one direct dependent (its parent crate) and is present in every toolchain its parent reaches. Procedural macros execute at build time on developer and CI machines. A package with code execution by design, one direct dependent, near-zero independent scrutiny, and presence in a large fraction of all builds is a supply-chain target profile, and dependent-count scoring cannot surface it even in principle: the count is one.

Result 3: the incumbent comparison

We mapped the ORACLE top-10 of crates.io, the watchlists above, and the threat-class list to their 2022 GitHub repositories (hand-curated mappings, published with the code) and checked membership in the incumbent's top-1000.

Of the crates ORACLE top-10, one appears: libc, at #257. serde — the most depended-upon crate in the registry by raw count — is absent. So are syn, proc-macro2, quote, cfg-if, and unicode-ident. The

threat-class list is absent in its entirety, 0 of 7. On the Debian side: OpenSSL is present at #42; zlib, libexpat, and libxml2 are absent.

Two distinct failures produce this, and they should not be conflated.

The scoring function fails on quiet finished infrastructure. The Pike score's inputs are contributor counts, organization counts, commit frequency, release cadence, issue activity, and mention counts. zlib is maintained software in its finished state: one maintainer, low commit frequency, rare releases, few issues. It scores near zero on every input while being, by any reasonable definition, among the most critical code in existence. This failure is intrinsic to fame-and-activity weighting and applies to every package in the tables above.

The collection pipeline fails on anything not hosted on GitHub. xz was hosted at git.tukaani.org in 2022. The pipeline enumerates GitHub repositories, so xz could not have appeared at any rank regardless of the scoring function. This is a coverage limitation, not a scoring limitation; it would have excluded xz even under a perfect score. We note it separately because it was the binding failure for the one package everyone in this field agrees was the catastrophe.

Limitations

Stated in full, because the comparison above is one-directional and the piece is descriptive throughout.

- **We did not run the symmetric test.** We checked whether ORACLE's head appears in the incumbent's list; we did not systematically check whether the incumbent's head (Linux, git, Node, Kubernetes) scores low on ORACLE. Most of the incumbent's top entries are applications rather than packages and have no node in a package dependency graph, so the symmetric test requires a corpus-mapping exercise we have not done. Until it is done, the correct statement is that ORACLE's head is invisible to the incumbent — not that the two rankings are anti-correlated.
- **One retrodiction is one retrodiction.** liblzma at #8 is a single post-hoc case, chosen because it is the consensus catastrophe. The metric's forward value is untested. The honest deployment model is a standing ranking whose future hits and misses accumulate in public.
- **A registered pooled retrodiction against RustSec advisories went against us, and we report it.** After this piece was drafted we ran the natural stress test: metrics computed on the 2022 crates snapshot, labels = RustSec vulnerability advisories dated strictly after it (159 affected crates in-snapshot), share of labeled crates in each metric's top decile. Every graph metric crushes the random null (55–63% vs 10%), but ORACLE (0.554) **lost** to dependent count (0.629) and PageRank (0.598) at the registered headline cell. Label bias favors popular crates (advisories are filed where the scrutiny is) and was stated before the run — but the registration committed to reporting the outcome as a loss, so: at *pooled* advisory retrodiction, concentration does not beat volume. This piece's claim is complementary value at the divergent head, which is a different cell and remains untested either way; a reader should still know the pooled test went the other way (`05-rustsec.mjs`, registered, single run).
- **Global rank correlation with volume metrics is high** (Spearman 0.95–0.99 across full registries). The divergence is concentrated at the head of the ranking — which is where prioritization decisions are made, but a reader should not picture two unrelated orderings.

- **Library enrichment is by design.** ORACLE's head is almost entirely libraries and build plumbing. For growth or importance claims that would be a confound; for supply-chain risk it is the point — libraries and build-time code are the attack surface.
- **The incumbent artifact is the v1-era list.** The v2 pipeline integrates deps.dev dependent counts and might narrow some gaps, though dependent counts are still volume signals and unicode-ident's count is six.[1]
- **Mappings are hand-curated.** Package-to-repository mappings for the join are a table in the published script; errors in it are ours and correctable.

Recommendation, ending on the artifact

Concentration of reach should be a column in criticality dashboards, next to — not instead of — activity-based scores. The two metrics disagree on a specific, enumerable set of packages: quiet, finished, deeply embedded libraries and degree-one build-time plumbing. That set is small (the head of the ORACLE ranking), cheap to compute for any registry with a dependency graph, and contains the known catastrophic case at rank eight of sixty-three thousand, twenty-one months before anyone knew to look.

Rather than end on the ask, we end on the artifact. github.com/thefalsework/conemass contains the ORACLE top-1000 for Debian trixie (2025) and crates.io, computed 2026-09-02 and published as-is. The files are dated; every future incident either involves a package in them or it does not, and either outcome scores the metric in public. The CLI beside them runs on any dependency graph in seconds — `node conemass.mjs Cargo.lock --top 50` works directly on a Rust project's lockfile — with no dependencies of its own, under Apache-2.0. The rows to read are the ones where `oracle_rank` is far ahead of `dependents_rank`.

Artifacts and reproducibility

Two artifacts accompany this piece so that its claims can be used, not just checked:

- **Published rankings** (thefalsework/conemass, `rankings/`): dated top-1000 ORACLE rankings for Debian trixie (2025) and crates.io (2022), with the dependent-count comparison columns inline.
- **A standalone CLI** (same repo, Apache-2.0): a single zero-dependency Node script that takes any dependency graph — a `Cargo.lock` directly, an edge-list CSV of `dependent,dependency` pairs, or a JSON graph — handles cycles, and emits a deterministic ranking. A 100,000-node registry takes seconds. Run it on your own graph and inspect the rows where `oracle_rank` is far ahead of `dependents_rank`. The repo's eight-package `test-cargo.lock` reproduces the metric's whole argument in one command: unicode-ident ranks first with two direct dependents, above proc-macro2 with three.

`oracle-scanner/` also contains the four studies behind this piece (retrodiction, cap sweep, crates replication, incumbent join), their raw outputs, the incumbent CSV as retrieved, and the hand-curated mapping table. Dependency snapshots and their extraction scripts are in `debian-study/` and `software-study/`. Every script's expectations were written in its header before first execution; results are in dated postscripts.

[1] The v2 pipeline's published dataset (an `all.csv` on Google Cloud Storage) was unavailable at retrieval time: the bucket returns "the billing account for the owning project is disabled." We used the most recent obtainable artifact of the score as consumed.

Disclosure. Drafting was AI-assisted under direction.